

Άσκηση 6

DNS ΚΑΚΟΒΟΥΛΕΣ ΕΠΙΘΕΣΕΙΣ ΤΥΠΟΥ
HIJACKING/PHARMING

Όνομα: Βασίλης Κουμεντέας
AM: 1083637 |

1. Υλοποίηση Εικονικής Μηχανής στο περιβάλλον Azure.

Χρησιμοποιήσαμε βήμα βήμα τις οδηγίες που μας παραδώσατε και δημιουργήσαμε την εικονική μηχανή Azure. Έπειτα, συνδεόμαστε με το private κλειδί που φτιάξαμε.

Ερ.1 Βασικές ρυθμίσεις ασφάλειας και πρόσβασης

- Χρησιμοποιώντας την εντολή `sudo passwd root`, βάζουμε κωδικό στον root χρήστη.
- Με την χρήση της εντολή `passwd`, βάζουμε κωδικό στον χρήστη που χρησιμοποιούμε εκείνη την στιγμή
- Εκτελούμε την εντολή `sudo nano /etc/ssh/sshd_config` και αλλάζουμε τις παρακάτω ρυθμίσεις:
 - `PasswordAuthentication yes`
 - `PubkeyAuthentication yes`
 - `ChallengeResponseAuthentication no`

Όμως, επειδή δεν είναι αρκετό, πρέπει να μπούμε στο αρχείο `/etc/ssh/sshd_config.d/50-cloud-init.conf` με την εντολή `sudo nano /etc/ssh/sshd_config.d/50-cloud-init.conf`. Εκεί θα βρούμε ένα δεύτερο `PasswordAuthentication` που πρέπει να κάνουμε `no->yes`, για να δουλέψει το ssh με κωδικό. Τέλος εκτελούμε την εντολή `sudo systemctl restart ssh` για να ενεργοποιηθούν οι αλλαγές μας.

Ερ.2 Παραμετροποίηση iptables

Για να χτίσουμε τους κανόνες, που μας ζητούνται, στο firewall, θα εκτελέσουμε τις παρακάτω εντολές με τη σειρά που μας ζητούνται:

- `sudo iptables -A INPUT -m state --state RELATED,ESTABLISHED -j ACCEPT`
- `sudo iptables -A INPUT -p tcp --dport 22 -s 150.140.0.0/16 -j ACCEPT`
- `sudo iptables -A INPUT -p udp --dport 53 -j ACCEPT`
- `sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT`
- `sudo iptables -A INPUT -i lo -j ACCEPT`
- `sudo iptables -P INPUT DROP → sudo iptables -P FORWARD DROP`
- `sudo iptables -P OUTPUT ACCEPT`
- `sudo apt-get install iptables-persistent → sudo netfilter-persistent save → sudo netfilter-persistent reload`
- Για να δοκιμάσουμε αν η θύρα 53 είναι ανοικτή για UDP πακέτα, μπορούμε να χρησιμοποιήσουμε τα εργαλεία `dig` (εγκαθιστώντας τα `dnsutils`) πχ με την εντολή `dig @localhost example.com`, `netcat` πχ με την εντολή `nc -vzu <localhost_ip> 53` και με την `tcpdump` με την εντολη `sudo tcpdump -i any udp port 53`.
- Για να δοκιμάσουμε αν η θύρα 443 είναι ανοικτή για TCP πακέτα, μπορούμε να χρησιμοποιήσουμε τα εργαλεία `telnet` με την εντολή `telnet <localhost_ip> 443`, `netcat` με την εντολή `nc -vz <localhost_ip> 443` και με το `tcpdump` με την εντολή `sudo tcpdump -i any tcp port 443`.

- Για να δούμε αν η θύρα 22 έχει τις παραπάνω ρυθμίσεις, μπορούμε να το κάνουμε με τα εξής εργαλεία: με το εργαλείο iptables (με την εντολή `sudo iptables -L`), με το εργαλείο ssh (με την εντολή `ssh <username>@<vm_ip>` ή με το `tcpdump` (με την εντολή `sudo tcpdump -i any tcp port 22`) και παρακολουθώντας τις ip που συνδέονται να δούμε αν είναι αυτές που θέλουμε εμείς.
- Αυτό μπορούμε να το καταφέρουμε χρησιμοποιώντας το εργαλείο `tcpdump`. Για να παρακολουθήσουμε όλα τα πακέτα που φτάνουν στην μηχανή, εκτελούμε την εντολή `sudo tcpdump -i any`. Και γενικά αν θέλουμε να είμαστε πιο συγκεκριμένοι μπορούμε να προσθέσουμε τις εντολές `sudo tcpdump -i any <protocol> port <port>`, όπως αναφέραμε και προηγουμένως.
- Το τελικό αποτέλεσμα του firewall μας, είναι αυτό:

```
up1083637@up1083637-debian11:~$ sudo iptables -L
Chain INPUT (policy DROP)
target     prot opt source                destination            state RELATED,ESTABLISHED
ACCEPT     tcp  --  150.140.0.0/16         anywhere               tcp dpt:ssh
ACCEPT     udp  --  anywhere              anywhere              udp dpt:domain
ACCEPT     tcp  --  anywhere              anywhere              tcp dpt:https
ACCEPT     all  --  anywhere              anywhere
ACCEPT     tcp  --  ppp046176209151.access.hol.gr anywhere              tcp dpt:ssh

Chain FORWARD (policy DROP)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
up1083637@up1083637-debian11:~$
```

(Να σχολιάσω, ότι το `tcp ppp046...` το έβαλα επιπλέον για την ip του σπιτιού μου, διότι, λόγω κάποιων γεγονότων, έπρεπε να το κάνω, αλλά δεν το έχω κάνει να παραμένει στο firewall.)

Ερ.3 Παραμετροποίηση fail2ban

- Εγκαθιστούμε το εργαλείο fail2ban με την εντολή `sudo apt install fail2ban` → `sudo systemctl enable fail2ban` → `sudo systemctl start fail2ban`. Για να επιβεβαιώσουμε ότι το fail2ban είναι ενεργό, χρησιμοποιούμε την εντολή `sudo systemctl status fail2ban`. Για να δοκιμάσουμε το ssh jail, χρησιμοποιούμε την εντολή `sudo fail2ban-client status sshd`. Το αποτέλεσμα, για να είναι σωστό, πρέπει να είναι αυτό:

```
up1083637@up1083637-debian11:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|   |- Currently failed: 0
|   |- Total failed:    0
|   `-- File list:      /var/log/auth.log
- Actions
  |- Currently banned: 0
  |- Total banned:    0
  `-- Banned IP list:
```

- Για να ρυθμίσουμε την fail2ban να ενεργοποιείται στις 5 αποτυχημένες προσπάθειες κωδικού ssh και να αποκλείει την IP για 10 λεπτά, μπαίνουμε στο αρχείο `jail.local`

(που είναι copy του jail.conf, το οποίο δεν μπορούμε να αλλάξουμε διότι θα χαλάσει το fail2ban) με την εντολή `sudo nano /etc/fail2ban/jail.local` και αλλάζοντας τα `bantime = 10m` και `maxretry = 5`. Τέλος, κάνουμε `sudo systemctl restart fail2ban` για να ενεργοποιηθούν οι αλλαγές (Να σχολιάσουμε ότι αυτές είναι οι *default* ρυθμίσεις του `jail.local`, οπότε πρακτικά δεν χρειάζεται να κάνουμε καμία αλλαγή).

```
# "bantime" is the number of seconds that a host is banned.
bantime = 10m

# A host is banned if it has generated "maxretry" during the last "findtime"
# seconds.
findtime = 10m

# "maxretry" is the number of failures before a host get banned.
maxretry = 5
```

- Δημιουργούμε νέο φίλτρο στο αρχείο `/etc/fail2ban/filter.d` με το όνομα `ssh-cert.conf`. Το καταφέρνουμε αυτό με την εντολή `sudo nano /etc/fail2ban/filter.d/ssh-cert.conf` και μέσα βάζουμε τα παρακάτω:

```
GNU nano 5.4 /etc/fail2ban/filter.d/ssh-cert.conf
[ssh-cert]
before = common.conf

[Definition]

_daemon = sshd

failregex = ^up1083637-debian11 sshd\[.*\]: Failed publickey for up1083637 from <HOST> port (?P<port>\d+) ssh2: RSA SHA256:(?P<fingerprint>[A-Za-z0-9+=]+)$

mode = normal
logpath = /var/log/auth.log
journalmatch = _SYSTEMD_UNIT=sshd.service + _COMM=sshd
```

(Να Σημειωθεί ότι μπορεί κάποιες από τις τελευταίες γραμμές, ίσως να μην χρειάζεται). Το `common.conf` είναι ένα set από κανόνες που έχει η fail2ban σε περίπτωση που θέλει κάποιος να φτιάξει νέο φίλτρο.

Στην συνέχεια πηγαίνουμε στο `jail.local` και προσθέτουμε το νέο jail του φίλτρου μας:

```
[ssh-cert]
enabled = true
port = ssh
filter = ssh-cert
logpath = /var/log/auth.log
backend = auto
bantime = 10m
maxretry = 1
findtime = 10m
```

Επίσης, πρέπει να μπούμε και στο αρχείο `/etc/fail2ban/jail.d/defaults-debian.conf` και να προσθέσουμε και εκεί το νέο μας jail:

```
GNU nano 5.4 /etc/fail2ban/jail.d/defaults-debian.conf
[sshd]
enabled = true
[ssh-cert]
enabled = true
```

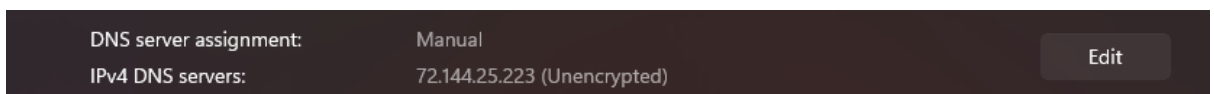
(Το τελευταίο ίσως να μην είναι αναγκαίο να γίνει, αλλά στην μηχανή μου αυτό ήταν που έκανε την διαφορά) Τέλος, εκτελούμε `sudo systemctl restart fail2ban` για να ενεργοποιηθούν οι αλλαγές μας.

(Σχόλιο: Δεν κατάφερα να κάνει ban κάποια IP διότι δεν έχω φτιάξει σωστά το failregex. Δηλαδή, το regex ακολουθά τους κανόνες της fail2ban, αλλά δεν βρίσκει παρόμοια logs, ενώ διαβάζει στο σωστό σημείο)

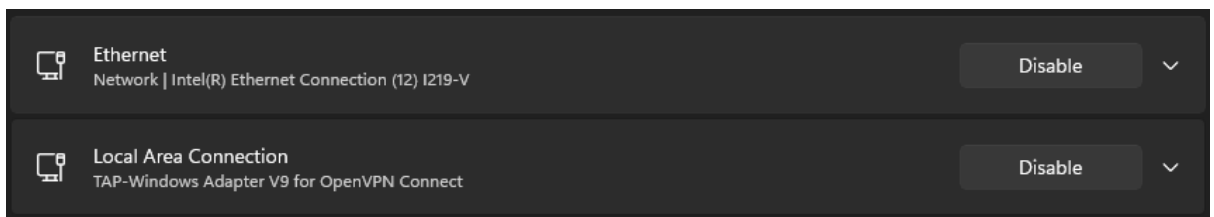
Ερ.4 Υλοποίηση DNS εξυπηρετητή

Στην μηχανή στο Azure, κατεβάζουμε το λογισμικό bind9 που υλοποιεί την DNS υπηρεσία με την εντολή `sudo apt install bind9 bind9utils`.

- Πηγαίνουμε στις ρυθμίσεις δικτύου στον προσωπικό μας υπολογιστή και αλλάζουμε τον διακομιστή DNS του υπολογιστή να κοιτάει στην εικονική μηχανή του Azure που έχουμε δημιουργήσει.



Αυτό το κάνουμε, και για την σύνδεση στο διαδίκτυο του υπολογιστή μας, και στις ρυθμίσεις δικτύου που είναι για VPN.



Οπότε τώρα με την εντολή `nslookup` μπορούμε να δούμε τον DNS server που χρησιμοποιούμε:

```
PS C:\WINDOWS\system32> nslookup
Default Server: UnKnown
Address: 72.144.25.223
```

Τώρα, για να ενεργοποιήσουμε τις καταγραφές των queries στον DNS server μας χρησιμοποιούμε την εντολή `sudo rndc querylog on`. Τα queries καταγράφονται αυτόματα στο `/var/log/syslog`:

```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> nslookup www.example.com
Server:      Unknown
Address:     72.144.25.223

Name:       www.example.com
Address:    150.140.139.251

PS C:\WINDOWS\system32>
up1083637@up1083637-debian11:~$ sudo tail -f /var/log/syslog
Dec 22 03:20:55 up1083637-debian11 waagent[685]: Zeroing chain 'OUTPUT'
Dec 22 03:20:55 up1083637-debian11 waagent[685]: ]
Dec 22 03:20:55 up1083637-debian11 waagent[685]: 2024-12-22T03:20:55.128110Z INFO ExtHandler ExtHandler [HEARTBEAT] Agent
t WAlinuxAgent-2.2.47 is running as the goal state agent
Dec 22 03:22:03 up1083637-debian11 dhclient[473]: XMT: Solicit on eth0, interval 121570ms.
Dec 22 03:24:05 up1083637-debian11 dhclient[473]: XMT: Solicit on eth0, interval 130170ms.
Dec 22 03:26:15 up1083637-debian11 dhclient[473]: XMT: Solicit on eth0, interval 109060ms.
Dec 22 03:26:27 up1083637-debian11 named[613]: client @0x7f3a5c593020 194.55.186.234#7792 (sl): query: sl IN ANY +E(0)DK
(10.1.1.4)
Dec 22 03:26:27 up1083637-debian11 named[613]: client @0x7f3a5c593020 194.55.186.234#7792 (sl): query (cache) 'sl/ANY/IN
' denied
Dec 22 03:26:27 up1083637-debian11 named[613]: client @0x7f3a5c593020 194.55.186.234#7792 (sl): query failed (REFUSED) f
or sl/IN/ANY at query.c:5561
Dec 22 03:28:04 up1083637-debian11 dhclient[473]: XMT: Solicit on eth0, interval 113260ms.
Dec 22 03:28:14 up1083637-debian11 named[613]: client @0x7f3a5c593020 150.140.254.58#63200 (223.25.144.72.in-addr.arpa):
query: 223.25.144.72.in-addr.arpa IN PTR + (10.1.1.4)
Dec 22 03:28:14 up1083637-debian11 named[613]: client @0x7f3a5c593020 150.140.254.58#63200 (223.25.144.72.in-addr.arpa):
query (cache) '223.25.144.72.in-addr.arpa/PTR/IN' denied
Dec 22 03:28:14 up1083637-debian11 named[613]: client @0x7f3a5c593020 150.140.254.58#63200 (223.25.144.72.in-addr.arpa):
query failed (REFUSED) for 223.25.144.72.in-addr.arpa/IN/PTR at query.c:5561
Dec 22 03:28:14 up1083637-debian11 named[613]: client @0x7f3a5c593020 150.140.254.58#63201 (www.example.com): query: www
.example.com IN A + (10.1.1.4)
Dec 22 03:28:14 up1083637-debian11 named[613]: client @0x7f3a5c593020 150.140.254.58#63202 (www.example.com): query: www
.example.com IN AAAA + (10.1.1.4)
```

Παρατηρούμε ότι στα logs καταγράφεται η IP του προσωπικού υπολογιστή μας, όπως επίσης και το domain name για το οποίο έγινε το query, την ημερομηνία και ώρα του query, την local IP του DNS, τον τύπο της κάθε καταγραφής, την Public IP του DNS μέσω της ανάστροφης ζώνης.

(Σχόλιο: Έκανα ότι μπορούσα για να κάνω το `dnsleaktest` να δουλέψει, αλλά δεν κατάφερα ποτέ να κάνω κανένα browser να βλέπει στο DNS του υπολογιστή μου. Οπότε το έκανα με `nslookup` που περνάει από τον DNS μας.)

- Στο αρχείο `/etc/bind/named.conf.options` προσθέτουμε την γραμμή `dump-file "/var/cache/bind/dump.db";`

```
GNU nano 5.4 /etc/bind/named.conf.options
options {
    directory "/var/cache/bind";
    dump-file "/var/cache/bind/dump.db";
    // If there is a firewall between you and nameservers you want
    // to talk to, you may need to fix the firewall to allow multiple
    // ports to talk.  See http://www.kb.cert.org/vuls/id/800113

    // If your ISP provided one or more IP addresses for stable
    // nameservers, you probably want to use them as forwarders.
    // Uncomment the following block, and insert the addresses replacing
    // the all-0's placeholder.

    // forwarders {
    //     0.0.0.0;
    // };
}
```

Στο αρχείο `/etc/bind/named.conf` προσθέτουμε τις παρακάτω ζώνες:

```
GNU nano 5.4 /etc/bind/named.conf
// This is the primary configuration file for the BIND DNS server named.
//
// Please read /usr/share/doc/bind9/README.Debian.gz for information on the
// structure of BIND configuration files in Debian, *BEFORE* you customize
// this configuration file.
//
// If you are just adding zones, please do that in /etc/bind/named.conf.local

include "/etc/bind/named.conf.options";
include "/etc/bind/named.conf.local";
include "/etc/bind/named.conf.default-zones";

zone "example.com"{
    type master;
    file "/var/cache/bind/example.com.db";
};

zone "0.168.192.in-addr.arpa" {
    type master;
    file "/var/cache/bind/192.168.0";
};
```

Και δημιουργούμε το αρχείο `/var/cache/bind/example.com.db`:

```
GNU nano 5.4 /var/cache/bind/example.com.db
$TTL 3D
@      IN      SOA      ns.example.com. admin.example.com. (
2008111001
      8H
      2H
      4W
      1D)

@      IN      NS       ns.example.com.
@      IN      MX       10 mail.example.com.

www    IN      A        150.140.139.251
mail   IN      A        192.168.0.102
ns     IN      A        192.168.0.10
*.example.com. IN      A 192.168.0.100
```

Στο οποίο φαίνεται ότι αλλάξαμε την IP του www.example.com να πηγαίνει στην IP 150.140.139.251 και όχι στην αρχική της. Μπορούμε να το ελέγξουμε και με την εντολή `nslookup www.example.com` (διότι είμαι σε windows):

```
PS C:\WINDOWS\system32> nslookup www.example.com
Server:      UnKnown
Address:     72.144.25.223

Name:   www.example.com
Address: 150.140.139.251

PS C:\WINDOWS\system32>
```

Μετά από αυτήν την αλλαγή, όταν πάμε στο www.example.com, μας πετάει στο photonics lab της σχολής CEID.



Photonics Lab

The **Photonic Networks and Technology Laboratory** aims at the development of all the critical technology for delivering advanced fiber optic components, tunable devices for high-capacity optical networks and fiber sensors. The Photonic technology laboratory is a well-equipped laboratory. Its infrastructure includes 40 Gb/s electrical and lightwave test & measurement equipment, a state-of-the art C-band DWDM test-bed and a 10Gb/s, programmable GMPLS testbed. We are also working on fiber sensors and tunable devices either biomedical or telecom applications. PNET lab is part of **Research Unit 1 of Computer Technology Institute** (<http://ru1.cti.gr/>)



(Σχόλια: Επειδή δεν το κατάφερα με τον τρόπο που ζητούσατε, το έβαλα στο αρχείο hosts να κάνε redirect στην IP που μας δώσατε για να δούμε που μας πάει)

- Acl “allowed”: το acl σημαίνει access control list και είναι ένας μηχανισμός για να δημιουργήσεις ομάδες IP διευθύνσεων με το όνομα allowed, τις οποίες θα χρησιμοποιεί σε παρακάτω ρυθμίσεις.

Recursion yes/no: ελέγχει αν ο DNS server θα εκτελεί recursive ερωτήσεις για λογαριασμό των clients.

Allow-recursion { localhost; allowed; }:: καθορίζει ποιοι clients μπορούν να κάνουν recursive queries.

listen-on port #nport { ips; ; }:: καθορίζει ποιες IP διευθύνσεις και θύρες θα ακούει ο DNS server για αιτήσεις.

Allow-query { localhost; allowed; }:: Καθορίζει ποιοι clients μπορούν να στείλουν DNS queries.

Allow-transfer { none; }:: Ελέγχει αν επιτρέπονται οι μεταφορές ζωνών σε άλλους DNS servers.

Forwarders { }:: Δηλώνει τους upstream DNS server στους οποίους θα προωθούνται οι ερωτήσεις που δεν μπορεί να απαντήσει ο διακομιστής.

Listen-on-v6 { any; }:: Καθορίζει ποιες IPv6 διευθύνσεις θα ακούει ο DNS server

Στο αρχείο named.conf.options προσθέτουμε τις παρακάτω ρυθμίσεις:

```
forwarders {
    8.8.8.8;
    8.8.4.4;
};

querylog yes;
recursion yes;
allow-recursion {localhost; 150.140.0.0/16; };
allow-query {localhost; 150.140.0.0/16; };
allow-transfer {none;};

listen-on port 53 {any;};

//=====
// If BIND logs error messages about the root key being expired,
// you will need to update your keys. See https://www.isc.org/bind-keys
//=====
dnssec-validation auto;

listen-on-v6 { any; };
```

(Σχόλια: Το acl “allowed” δεν το έβαλα γιατί μου έκανε fail την bind9)

- Εκτελώντας την εντολή `sudo tcpdump -i eth0 port 22` βλέπουμε όλα τα πακέτα που περνάνε από τη θύρα ssh. Κάνοντας ένα πείραμα ζητώντας από έναν συνάδελφό με IP 150.140.254.90 να συνδεθεί στη μηχανή μας με ssh βλέπουμε τα πακέτα για τη σύνδεση παρακάτω:

```
19:58:11.849022 IP 150.140.254.142.61080 > up1083637-debian11.internal.cloudapp.net.ssh: Flags [.], ack 63700, win 251, length 0
19:58:11.899336 IP 150.140.254.142.61080 > up1083637-debian11.internal.cloudapp.net.ssh: Flags [.], ack 64096, win 255, length 0
19:58:11.930069 IP 150.140.254.90.7282 > up1083637-debian11.internal.cloudapp.net.ssh: Flags [S], seq 3075979484, win 64240, options [mss 1328,nop,wscale 8,nop,nop,sackOK], length 0
19:58:11.930121 IP up1083637-debian11.internal.cloudapp.net.ssh > 150.140.254.90.7282: Flags [S.], seq 2459757523, ack 3075979485, win 64240, options [mss 1460,nop,nop,sackOK,nop,wscale 7], length 0
19:58:11.939927 IP 150.140.254.142.61080 > up1083637-debian11.internal.cloudapp.net.ssh: Flags [.], ack 64548, win 254, length 0
19:58:11.949799 IP up1083637-debian11.internal.cloudapp.net.ssh > 150.140.254.142.61080: Flags [P.], seq 64548:64728, ack 37, win 501, length 180
19:58:11.949881 IP up1083637-debian11.internal.cloudapp.net.ssh > 150.140.254.142.61080: Flags [P.], seq 64728:64944, ack 37, win 501, length 216
19:58:11.950128 IP up1083637-debian11.internal.cloudapp.net.ssh > 150.140.254.142.61080: Flags [P.], seq 64944:65432, ack 37, win 501, length 488
19:58:11.985576 IP 150.140.254.90.7282 > up1083637-debian11.internal.cloudapp.net.ssh: Flags [.], ack 1, win 1027, length 0
19:58:11.988541 IP 150.140.254.90.7282 > up1083637-debian11.internal.cloudapp.net.ssh: Flags [P.], seq 1:34, ack 1, win 1027, length 33: SSH: SSH-2.0-OpenSSH_for_Windows_9.5
19:58:11.988563 IP up1083637-debian11.internal.cloudapp.net.ssh > 150.140.254.90.7282: Flags [.], ack 34, win 502, length 0
19:58:11.991543 IP up1083637-debian11.internal.cloudapp.net.ssh > 150.140.254.142.61080: Flags [P.], seq 65432:65980, ack 37, win 501, length 548
19:58:11.997969 IP up1083637-debian11.internal.cloudapp.net.ssh > 150.140.254.90.7282: Flags [P.], seq 1:41, ack 34, win 502, length 40: SSH: SSH-2.0-OpenSSH_8.4p1 Debian-5+deb11u3
19:58:12.002912 IP 150.140.254.142.61080 > up1083637-debian11.internal.cloudapp.net.ssh: Flags [.], ack 64944, win 252, length 0
19:58:12.044644 IP 150.140.254.142.61080 > up1083637-debian11.internal.cloudapp.net.ssh: Flags [.], ack 65980, win 255, length 0
```

- Τροποποιούμε το αρχείο C:\Windows\System32\drivers\etc\hosts ως εξής:

```
$ hosts X
C: > Windows > System32 > drivers > etc > $ hosts
6  # entry should be kept on an individual line. The IP address should
7  # be placed in the first column followed by the corresponding host name.
8  # The IP address and the host name should be separated by at least one
9  # space.
10 #
11 # Additionally, comments (such as these) may be inserted on individual
12 # lines or following the machine name denoted by a '#' symbol.
13 #
14 # For example:
15 #
16 #      102.54.94.97      rhino.acme.com      # source server
17 #      38.25.63.10      x.acme.com          # x client host
18
19 # localhost name resolution is handled within DNS itself.
20 #   127.0.0.1      localhost
21 #   ::1            localhost
22 150.140.129.14 www.example.com
23
```

Και τώρα όταν πάμε να μπούμε στο www.exmaple.com μας πετάει στο eclass.upatras.gr.

- Στον δοσμένο κώδικα, άλλαξα τα sourceIP = “150.140.254.142” όπου είναι η διεύθυνση του επιτιθέμενου, destIP = “72.144.25.223” όπου είναι η διεύθυνση του θύματος, spoofing_set = range(30000, 40000) όπου είναι τα transaction Ids που θα χρησιμοποιήσει για να επιτεθεί, victim_host_name = www.example.com το domain που θέλουμε να αλλάξουμε, rougeIP = “142.251.209.46” όπου είναι η διεύθυνση που θα κάνει redirect ο dns αντί του domain www.exmaple.com.

Ο λόγος που αποτυγχάνουν τα πακέτα είναι λόγο των λανθασμένων transaction ID. Ο κώδικας προσπαθεί να μαντέψει τα transaction Ids, οπότε μόνο αν καταφέρει και πετύχει σωστό ID θα καταφέρει να περάσει το μήνυμα, αλλιώς απορρίπτεται. Όσο μεγαλύτερο spoofing_set έχουμε, τόσο μεγαλώνουμε την πιθανότητα επιτυχίας να περάσει το μήνυμα.